

Assignment 2 (Weeks 3 and 4)

Part I: Short Answer / Reasoning Questions

Q1. NULL comparison in WHERE clause.

The reason this query always returns 0 is that NULL isn't really a "value" in the way 5 or "abc" are values – it's SQL's way of saying "I don't know what this is." So when you write `phone_number = NULL`, you're not asking "is this column empty," you're asking "does this unknown thing equal this other unknown thing," and SQL can't say yes to that. It can't even say no. It just shrugs and returns UNKNOWN, and WHERE only keeps rows where the condition is TRUE. UNKNOWN gets thrown out along with the FALSE rows, which is the part that trips people up because it feels like it should count those students, but it doesn't.

The fix is to use `IS NULL` instead, since that's a special predicate built specifically to check for nulls and actually gives you a real TRUE/FALSE:

```
SELECT COUNT(*)
FROM Student
WHERE phone_number IS NULL;
```

More generally, the reason NULL gets excluded from aggregates and comparisons comes down to the same idea – databases use three-valued logic (true, false, unknown) instead of just true/false, because "missing data" genuinely isn't the same thing as "false." If AVG or SUM tried to include a NULL salary in the calculation, what would it even add? You can't average in a number you don't have, so the engine just skips those rows for that particular column. (COUNT(*) is the odd one out here since it's literally just counting rows that exist, not evaluating any column's value, so NULLs don't bother it.)

Q2. Is DISTINCT redundant with GROUP BY department?

Yeah, the student's right here, DISTINCT isn't doing anything in this query.

Think about what GROUP BY already does on its own: it takes the whole table and squashes it down into one row per department before SELECT even runs. So by the time we're picking columns to display, there's already exactly one row per department value – there's no way for two output rows to both say "Sales" because GROUP BY already merged all the Sales rows together. COUNT(*) is an aggregate too, so it's also just one number per group. Since both columns in the SELECT list are already guaranteed unique per row, slapping DISTINCT on top is just asking the database to deduplicate a result that has no duplicates to remove. It would matter if there were some other non-aggregated column thrown in that could repeat within a group, but that's not what's happening here. Worst case, it just costs a bit of extra processing for nothing.

Q3. INNER JOIN vs LEFT JOIN row counts.

If the LEFT JOIN comes back with more rows than the INNER JOIN, that gap is basically telling you "some customers exist who have zero orders." You don't need to look at the data to know that – it falls right out of how the two joins behave.

INNER JOIN only keeps a row when both sides actually match up, so any customer who's never placed an order just vanishes from that result – there's nothing on the Order side for them to pair with. LEFT JOIN, on the other hand, refuses to drop anyone from the left table (Customer), so even a customer with no matching orders still shows up, just with NULLs sitting in all the Order columns. So the scenario causing the gap is exactly that: one or more rows in Customer have no corresponding `customer_id` anywhere in Order. And actually, the size of that gap tells you something too – it's roughly the number of customers who've never ordered (assuming nothing weird is going on with duplicate matches on the other side).

Q4. Invalid GROUP BY query with employee_name.

This one's invalid because `employee_name` just shouldn't be allowed to sit in the SELECT list like that. We're grouping by department, which means each output row now represents a whole bunch of employees squashed into one group – and a department obviously has more than one employee. So which `employee_name` is SQL supposed to print? There isn't a single correct answer, which is exactly the problem.

If you really sit and think about what it would mean for this to "work," the database would have to just pick *some* employee's name out of the group arbitrarily – maybe whoever happens to be first in storage order, maybe the last one inserted, who knows. That's not really a query result anymore, that's a coin flip dressed up as data. Standard SQL doesn't let this happen for that reason (older MySQL versions actually did allow it and just picked something silently, which is honestly worse, since you'd get an answer that looks legitimate but means nothing).

If we genuinely just want department-level averages, drop the name column:

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department;
```

But if the actual goal is to see every employee's salary next to their department's average side by side, GROUP BY is the wrong tool entirely – a window function handles that without collapsing anything:

```
SELECT department,
       employee_name,
       salary,
       AVG(salary) OVER (PARTITION BY department) AS dept_avg_salary
FROM Employee;
```

Q5. WHERE with AVG(salary) – why it fails.

The bug is putting `AVG(salary)` inside WHERE. The thing people forget is that SQL doesn't run top-to-bottom the way it's written – there's a logical order it actually follows behind the scenes, and WHERE happens way before grouping does. Roughly:

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

So at the exact moment WHERE is being evaluated, the database is still looking at raw, individual Employee rows one at a time – the groups haven't formed yet, which means there's no such thing as "the average salary of this group" yet either. `AVG(salary)` simply doesn't exist at that stage of execution, so asking WHERE to filter on it is asking for something that hasn't been computed.

HAVING exists specifically to fix this – it runs after the groups and their aggregates are ready, so it's the right place to filter on AVG.

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 80000;
```

Q6. Correlated vs non-correlated subqueries.

(a) **Products priced above the overall average product price.** This one's non-correlated. The average price across the whole Product table is just one fixed number – it doesn't change depending on which product row you're currently looking at, so there's nothing for the inner query to "correlate" to. It runs once, gets one number back, and every outer row gets compared against that same number.

```
SELECT *
FROM Product
WHERE price > (SELECT AVG(price) FROM Product);
```

(b) **Employees who earn more than the average salary of their own department.** This time it has to be correlated, because "their own department" means the threshold changes depending on who you're looking at. An employee in Sales gets compared to Sales's average, an employee in Engineering gets compared to Engineering's average – so the inner query needs to know which outer row it's currently being run for, which is what the correlation condition does.

```
SELECT e1.*
FROM Employee e1
WHERE e1.salary > (
    SELECT AVG(e2.salary)
    FROM Employee e2
    WHERE e2.department = e1.department
);
```

Performance-wise, the difference is pretty significant once tables get big. A non-correlated subquery runs one time, full stop, and that single answer gets reused for everything. A correlated one conceptually has to re-run for every row (or at least once per distinct department, if the optimizer is being clever) coming from the outer query. So if the outer table has a few million rows, that correlated subquery could turn into something close to $n \times m$ work instead of just m . This is usually why people end up rewriting correlated subqueries as a join against a pre-aggregated derived table when performance starts to matter.

Part II: Long Answer Question

Schema recap.

```
Customer(customer_id, name, email, phone)
Restaurant(restaurant_id, name, cuisine_type, rating)
MenuItem(item_id, restaurant_id, item_name, price)
Order(order_id, customer_id, restaurant_id, order_date, status)
OrderItem(order_id, item_id, quantity)
```

(a) Customers who have never placed an order

```
SELECT c.customer_id, c.name
FROM Customer c
LEFT JOIN Order o
    ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

The reasoning here is the same idea as Q3 above – LEFT JOIN keeps every customer no matter what, even the ones with nothing on the Order side, and fills those missing columns with NULL. So filtering on `o.order_id IS NULL` afterward catches exactly the customers who never matched anything, which is what “never placed an order” actually means. An INNER JOIN would be the wrong call entirely – it would just quietly drop those customers instead of showing them, which is the opposite of what we want. I also like NOT EXISTS as an alternative since it reads almost like English and avoids the weird edge cases NOT IN can run into with NULLs:

```
SELECT c.customer_id, c.name
FROM Customer c
WHERE NOT EXISTS (
    SELECT 1
    FROM Order o
    WHERE o.customer_id = c.customer_id
);
```

(b) Average order value per restaurant (orders > 5)

```
SELECT o.restaurant_id,
       AVG(order_totals.order_value) AS avg_order_value
FROM Order o
JOIN (
    SELECT oi.order_id, SUM(mi.price * oi.quantity) AS order_value
    FROM OrderItem oi
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY oi.order_id
) AS order_totals
ON o.order_id = order_totals.order_id
GROUP BY o.restaurant_id
HAVING COUNT(DISTINCT o.order_id) > 5;
```

The tricky part of this one is that “order value” isn’t sitting in any single table – it’s something you have to build first by multiplying price by quantity and summing across all the items in an order.

So I compute that as its own derived table before doing anything else, because trying to average per restaurant and sum per order in the same single GROUP BY just doesn't work – those are two different levels of grouping happening at once. And the "more than 5 orders" condition has to live in HAVING, not WHERE, since it's filtering based on COUNT(DISTINCT order_id), which is an aggregate that only exists once the restaurant-level groups have formed.

(c) Restaurants whose average order value exceeds the platform-wide average

```
SELECT restaurant_avg.restaurant_id,
       restaurant_avg.avg_order_value
FROM (
    SELECT o.restaurant_id,
           AVG(order_totals.order_value) AS avg_order_value
    FROM Order o
    JOIN (
        SELECT oi.order_id, SUM(mi.price * oi.quantity) AS order_value
        FROM OrderItem oi
        JOIN MenuItem mi ON oi.item_id = mi.item_id
        GROUP BY oi.order_id
    ) AS order_totals
    ON o.order_id = order_totals.order_id
    GROUP BY o.restaurant_id
    HAVING COUNT(DISTINCT o.order_id) > 5
) AS restaurant_avg
WHERE restaurant_avg.avg_order_value > (
    SELECT AVG(order_totals.order_value)
    FROM Order o
    JOIN (
        SELECT oi.order_id, SUM(mi.price * oi.quantity) AS order_value
        FROM OrderItem oi
        JOIN MenuItem mi ON oi.item_id = mi.item_id
        GROUP BY oi.order_id
    ) AS order_totals
    ON o.order_id = order_totals.order_id
);
```

This one doesn't need to be correlated, and honestly making it correlated would actually be a mistake here, not just unnecessary. The "platform-wide average" is supposed to be one fixed number that every restaurant gets measured against – the second I let it reference the outer restaurant row, it stops being platform-wide and starts being something else entirely (and also gets recomputed pointlessly for every single restaurant instead of once). So I just nest my part (b) query inside a plain subquery and compare against it directly.

(d) Most ordered menu item (by total quantity) for each restaurant

```
SELECT ranked.restaurant_id,
       ranked.item_id,
       ranked.item_name,
       ranked.total_quantity
FROM (
    SELECT mi.restaurant_id,
           mi.item_id,
```

```

        mi.item_name,
        SUM(oi.quantity) AS total_quantity,
        RANK() OVER (
            PARTITION BY mi.restaurant_id
            ORDER BY SUM(oi.quantity) DESC
        ) AS rnk
    FROM MenuItem mi
    JOIN OrderItem oi ON mi.item_id = oi.item_id
    GROUP BY mi.restaurant_id, mi.item_id, mi.item_name
) AS ranked
WHERE ranked.rnk = 1;

```

I went with a window function here instead of trying to force this through plain GROUP BY, because RANK() lets me rank items inside each restaurant's own little group while still keeping item_id and item_name attached – then I just keep whatever lands at rank 1 per restaurant.

The actual limitation of a basic GROUP BY + MAX approach is something I ran into thinking about this: say you try `SELECT restaurant_id, MAX(total_quantity) ... GROUP BY restaurant_id` on top of the per-item totals – that gives you the highest quantity number per restaurant, sure, but it can't also tell you *which item* hit that number, because item_id and item_name are non-aggregated columns and GROUP BY has no legal way to attach them to a MAX() result (basically the same problem as Part I, Q4). You'd then need a second pass – some kind of self-join or subquery – to go match that max value back to the actual item row, and even that step can backfire and return two rows if two items happen to tie. Window functions just sidestep the whole mess in one shot, and you even get to choose how ties are handled by picking RANK vs ROW_NUMBER vs DENSE_RANK.

Part III: Normalization

Relation under review:

```
Enrollment(student_id, student_name, course_id, course_name,
            instructor_id, instructor_name, instructor_office,
            grade, enrollment_date)
```

Q1. Functional dependencies implied by the scenario

Going through the assumptions one at a time, here's what falls out:

- $student_id \rightarrow student_name$
- $course_id \rightarrow course_name$
- $course_id \rightarrow instructor_id$ (since a course only has one instructor)
- $instructor_id \rightarrow instructor_name$
- $instructor_id \rightarrow instructor_office$ (one instructor, one office)
- $course_id \rightarrow instructor_name, instructor_office$ – this one's transitive, it comes along for the ride through $instructor_id$
- $(student_id, course_id) \rightarrow grade, enrollment_date$

Q2. Is this table in 1NF?

I'd say yes, it's in 1NF – as long as we're assuming grade and the other fields each hold one single value (no comma-separated grades crammed into one cell or anything like that). Every column is atomic, and $(student_id, course_id)$ gives us a clean way to identify each row uniquely. 1NF is a pretty low bar, honestly – it just cares about atomic values and no repeating groups, it doesn't care at all about whether the table is full of redundant data, which is exactly where things start going wrong next.

Q3. Partial dependency violating 2NF

The primary key here is the composite $(student_id, course_id)$, and 2NF says every non-key column needs to depend on *both* parts of that key together, not just one half of it. Looking at the dependencies from Q1, a bunch of columns clearly fail that test:

- $student_id \rightarrow student_name$ – only needs $student_id$, $course_id$ is irrelevant to it
- $course_id \rightarrow course_name, instructor_id, instructor_name, instructor_office$ – only needs $course_id$, $student_id$ is irrelevant

So $student_name$ and all the course/instructor stuff are only depending on half the key – that's the partial dependency.

A concrete example of the anomaly: picture a course with 40 students enrolled in it. $course_name$, $instructor_id$, $instructor_name$, and $instructor_office$ would literally be copy-pasted across all 40 rows. Now if that instructor gets a new office, somebody has to go update all 40 rows – miss even one and now the data contradicts itself (update anomaly). If the last student drops the course and

that row gets deleted, the course's info just disappears from the database entirely, even though the course itself might still be running (deletion anomaly). And if a brand new course gets added but nobody's enrolled in it yet, there's literally no row to put it in, since every row requires a `student_id` (insertion anomaly).

Q4. Transitive dependency violating 3NF

3NF says non-key attributes shouldn't depend on the key *through* some other non-key attribute. That's exactly what's happening here:

$$\text{course_id} \rightarrow \text{instructor_id} \rightarrow \text{instructor_name}, \text{instructor_office}$$

`instructor_name` and `instructor_office` aren't really hanging off the primary key directly – they're hanging off `instructor_id`, which itself is just another regular column, not part of the key. That chain is the transitive dependency, and it's a 3NF violation.

Anomaly: if an instructor switches offices, every single row tied to any course that instructor teaches needs updating – same flavor of problem as Q3, just one level removed and now centered on the instructor instead of the course. There's also just plain redundant storage of the same instructor's name and office repeated over and over for every course they teach, which is wasted space at best and a consistency risk at worst.

Q5. Decomposition into 3NF

```
Student(student_id, student_name)
```

Pulled out because of: $\text{student_id} \rightarrow \text{student_name}$.

```
Instructor(instructor_id, instructor_name, instructor_office)
```

Pulled out because of: $\text{instructor_id} \rightarrow \text{instructor_name}, \text{instructor_office}$.

```
Course(course_id, course_name, instructor_id)
```

Pulled out because of: $\text{course_id} \rightarrow \text{course_name}, \text{instructor_id}$ (`instructor_id` here is just a foreign key pointing back at `Instructor`).

```
Enrollment(student_id, course_id, grade, enrollment_date)
```

Pulled out because of: $(\text{student_id}, \text{course_id}) \rightarrow \text{grade}, \text{enrollment_date}$ (`student_id` and `course_id` are foreign keys here, and together they're still the primary key of this table).

Primary keys: `student_id` in `Student`; `instructor_id` in `Instructor`; `course_id` in `Course`; the pair $(\text{student_id}, \text{course_id})$ in `Enrollment`.

Splitting it up this way kills both problems from before – the partial dependency is gone because every table's non-key columns now actually depend on that table's *whole* key, and the transitive dependency is gone because instructor info lives in its own table instead of hanging off `course_id` through an extra hop.

Part IV: Design Justification

In Part II(c), I had a real choice between a correlated and a non-correlated subquery for the platform-wide average, and I went with non-correlated on purpose. Making it correlated would've technically run, but it would have recomputed the exact same "platform average" number over and over for every restaurant, which is wasteful and also a little misleading – the whole point of a platform-wide figure is that it *doesn't* change per restaurant, so correlating it would contradict what the number is supposed to represent. It's also just easier to read this way – a plain subquery says "here's one fixed benchmark" clearly, whereas a correlated version might make someone reading it assume it depends on the outer row when it shouldn't.

For Part III, the trade-off with normalizing Enrollment is pretty real. Before splitting it up, one query could grab a student's name, their course, the instructor, and their grade with zero joins. After normalizing into Student, Instructor, Course, and Enrollment, that same lookup now needs three or four joins glued together, which is more to write and, on big datasets, slower to run. That's honestly why real systems sometimes don't fully normalize everything – for queries that get hit constantly (think dashboards or reports), some teams will deliberately keep a denormalized table or a materialized view around, trading a bit of redundancy for fewer joins and faster reads, even knowing it means more care is needed to keep that copy in sync.